

00 — อภิธานศัพท์ สัญลักษณ์ และตัวอย่างประจำชุดไม้ต 05–07

ไฟล์นี้คือ **ที่เดียว** ที่คำศัพท์ ตัวอย่าง สัญลักษณ์ คำคงที่ และเลขสมการของไม้ต 05, 06, 07 ถูกนิยาม · ไม้ตทั้งสามลิงก์มาที่นี่และ **ไม่นิยามซ้ำด้วยคำของตัวเอง** — ไม้ตมีหน้าที่ *สอน* (ภาพ · ที่มา · การ derive · ความหมาย) ส่วนไฟล์นี้มีหน้าที่ *นิยาม*

อ่านหัวข้อ 0 ก่อนหนึ่งครั้ง จากนั้นใช้เป็นตารางเปิดดู · ทุกระรกต์โค้ดที่อ้างเป็น file.py:NN ในไฟลเดอร์ tron1-r1-isaacrlab/ (ยกเว้นที่ระบุว่าเป็น IsaacLab/, torch/ หรือ ../ = รากของ study_tron1/) · cfg:NN ย่อจาก limx_rsl_r1_ppo_cfg.py:NN · limx_base_env_cfg.py หมายถึงไฟล์ใน cfg/SF/ เสมอ (PF/WF มีไฟล์ชื่อเดียวกัน แต่เนื้อหาต่างกัน) · :NN เดี่ยว ๆ = ไฟล์เดียวกับที่อ้างล่าสุดก่อนหน้าในแถว/ย่อหน้าเดียวกัน และตรวจกับไฟล์จริงแล้ว (2026-09-11) · ตัวเลขทุกตัวในคอลัมน์ "คำใน toy" คำนวณจาก ตัวอย่างในหัวข้อ 8 ด้วยสคริปต์เดียวกัน

0. วิธีอ่านตาราง และชนิดของวัตถุ 11 แบบ

ทุกตารางในไฟล์นี้ใช้คอลัมน์ชุดเดียวกัน:

คอลัมน์	บอกอะไร
คำ / สัญลักษณ์	ชื่อไทยก่อน · ชื่ออังกฤษในวงเล็บครั้งแรกที่พบ · สัญลักษณ์เขียนแบบ LaTeX
ชนิด	วัตถุนี้เป็นของประเภทไหน — เลือกจาก 11 แบบข้างล่างเท่านั้น
ความหมาย	หนึ่งประโยค
โค้ด	บรรทัดที่ประกาศ / บรรทัดที่ใช้ · วางถ้าเป็นแนวคิดลึ้น
คำใน toy	ค่าที่ช่อง e0 t0 ของตัวอย่างในหัวข้อ 8 เว้นแต่ระบุช่องอื่น · ถ้าค่าของ Tron1 จะวิ่งต่างออกไป จะเขียนต่อท้ายหลังจุดกลาง
ใช้ครั้งแรก	หัวข้อของไม้ต 05 (หลังเรียบเรียงใหม่) ที่ผู้อ่านจะเจอคำนี้ครั้งแรก
สูตร / รรรมเนียน	รูปปิดหนึ่งบรรทัดพร้อมเลขสมการ หรือหมายเหตุเรื่องรรรมเนียนการเขียน

ชนิดของวัตถุ — คำถามแรกของทุกสัญลักษณ์คือ *"มันเป็นของประเภทไหน"* เพราะสัญลักษณ์ ที่หน้าตาคล้ายกันมักเป็นคนละประเภท และประเภทเปลี่ยนไปเรื่อย ๆ ตามสาย:

พื จ กั ชั น → การแจกแจง → เวกเตอร์ → **scalar**

ชนิด	หมายถึง	ตัวอย่าง
พื จ กั ชั น	รับของเข้าไป คืนของออกมา ยังไม่ได้เรียก	$\pi_\theta, V_\phi, \mu_\theta$
การแจกแจง	ระดัหนึ่งใด — อีอบเจกต์ Noxmal ในโค้ด สุ่มได้ ตามความหนาแน่นได้	$\pi_\theta(\cdot \mid s)$
เวกเตอร์	ตัวเลขหลายตัวเรียงกัน มี shape	$\mu(8, \cdot), s(42, \cdot)$
scalar	ตัวเลขตัวเดียว	$\pi_\theta(a \mid s), L, \rho$
ค่าคงที่	ตัวเลขที่มนุษย์ตั้งใน config ไม่ถูกเรียนรู้	γ, ϵ, lr เริ่มต้น
โค้ด	บรรทัดหรือเมธอดที่ <i>ทำ</i> อะไรสักอย่าง ไม่มีค่าของตัวเอง	<code>.sample(), optimizer.step()</code>
หน่วยนับ	วิธีห้นข้อมูล/เวลา	step, episode, iteration
แนวคิด	ไม่มีค่า ไม่มีโค้ด เป็นคำอธิบายพฤติกรรม	on-policy, bootstrapping
เซต	กลุ่มของสมาชิกทั้งหมดที่เป็นไปได้ นอกนั้น ไม่สุ่ม ไม่คำนวณ	$\mathcal{S}, \mathcal{A}, \Delta(\mathcal{A})$
สมการ	ความสัมพันธ์ที่มีเลขกำกับ — ไม่ใช่ตัวเลข ไม่ใช่โค้ด	Bellman (4), SGD (12), กฎปรับ lr (37)
ตัวดำเนินการ	เครื่องหมายที่บอกว่าเขาจะอย่างไรบ้างทำอะไรกัน	$\odot, , \sim$

กฎการอ่านที่ใช้ทุกครั้ง — ถ้าสัญลักษณ์ตัวเดียวกันไฟล์ในสองชนิด (เช่น π_θ กับ $\pi_\theta(a \mid s)$) แปลว่ามัน *ไม่ใช่* ของเดียวกัน แต่เป็นสองจุดบนสายข้างบน — ดูหัวข้อ 7

1. ตัวอย่างและชื่อเฉพาะ

เรียงตามตัวอักษร · ทุกตัวถูกเขียนชื่อเต็มครั้งแรกที่ใช้ในไม้ตแต่ละเล่มด้วย

ตัวอย่าง	ย่อมาจาก	คือ
ADC	Analog-to-Digital Converter	ตัวแปลงสัญญาณเซนเซอร์เป็นตัวเลข — ใช้เป็นอุปมาในไม้ต 05 §2.5
Adam	<i>ไม่ใช่ตัวอย่าง</i> — ชื่อเรียกของ optimizer (มีที่มาจาก adaptive moment estimation)	กฎอัปเดต weight ที่ Tron1 ใช้ (ppo.py:85) · หัวข้อ 5
DDPG · TD3	Deep Deterministic Policy Gradient · Twin Delayed DDPG	อัลกอริทึม policy แบบกำหนดแน่ (deterministic) · ไม้ตอ้างเพื่อ เทียบ เท่านั้น
CLI	Command-Line Interface	บรรทัดคำสั่ง — ที่ที่ --num_envs 2048 ถูกพิมพ์กับคำปரியของ cfg
DP	Dynamic Programming	วิธีแก้ MDP แบบตารางที่รู้แบบจำลองของโลกครบ · ทางต้นข้อแรกในไม้ต 05 ภาค 3
EKF · KF	(Extended) Kalman Filter	ตัวประมาณสถานะสาย control · ไม้ต 05 §4.4 เทียบกับ encoder

ตัวย่อ	ย่อมาจาก	คือ
ELU	Exponential Linear Unit	ฟังก์ชันกระตุ้นที่ Tron1 ใช้ทุกชั้นซ่อน (activation="elu", cfg:90) · สมการ (11)
GAE	Generalized Advantage Estimation	วิธีประมาณ advantage ที่ Tron1 ใช้ · สมการ (25)–(26)
GPU · VRAM	Graphics Processing Unit · Video RAM	การ์ดจอและหน่วยความจำบนการ์ด — ที่ที่ทุกอย่างในหัวข้อ 4–5 อาศัยอยู่ตอนแรก
IMU	Inertial Measurement Unit	เซนเซอร์วัดความเร่งและอัตราหมุนของลำตัว — ที่มาของ base_ang_vel และ projected_gravity
KL	Kullback–Leibler divergence	ชื่อคนสอนคน ไม่ใช่ตัวย่อศัพท์ · ตัววัดว่าการแจกแจงสองอันต่างกันแค่ไหน · สมการ (30)
LQR	Linear Quadratic Regulator	ตัวควบคุมสาย control ที่ใช้เทียบกับ critic ในไม้ด 05 §4.3 (อ่านข้ามได้)
lr	learning rate	ก้าวยาวแค่ไหนต่อหนึ่งสเต็ป · เขียนแทนด้วย α ก็ได้ · หัวข้อ 5
MC	Monte Carlo	ประมาณค่าคาดหวังด้วยการรอฟลจจริงจนจบ · ไม้ด 05 §3.2
MDP · POMDP	(Partially Observable) Markov Decision Process	กรอบคณิตศาสตร์ของปัญหา · "PO" = มองเห็นสถานะไม่ครบ ซึ่งเป็นกรณีของ Tron1
MLP	Multi-Layer Perceptron	โครงข่ายประสาทแบบชั้นซ้อนธรรมดา (nn.Linear สลับกับ activation) — actor, critic, encoder ของ Tron1 ล้วนเป็น MLP
MSE	Mean Squared Error	ค่าเฉลี่ยของกำลังสองของส่วนต่าง — โครงของ value loss และ loss ของ encoder
NN	Neural Network	โครงข่ายประสาทเทียม
ONNX	Open Neural Network Exchange	รูปแบบไฟล์กลางสำหรับ export โมเดล (play.py:111–121)
PD	Proportional–Derivative controller	ตัวคุมมอเตอร์ขึ้นล่างสุด แปลงนูนเป้าหมายเป็นแรงบิด · สมการ (18)
PDF	Probability Density Function	ฟังก์ชันความหนาแน่นความน่าจะเป็น · สมการ (13)
PPO	Proximal Policy Optimization	อัลกอริทึมที่ Tron1 ใช้แทน "proximal" = ห้ามขยับไกลจากของเดิม · Schulman et al. 2017, arXiv:1707.06347
REINFORCE	ชื่อ (Williams 1992) ไม่ใช่ตัวย่อ	policy gradient แบบดั้งเดิมที่ไม่มี critic · ไม้ด 05 §4.1
RL	Reinforcement Learning	การเรียนรู้แบบเสริมกำลัง — เรียนจาการรางวัล ไม่ใช่จากเฉลย
RNG	Random Number Generator	ตัวสร้างเลขสุ่มของเครื่อง — ที่มาของ ϵ
SAC	Soft Actor-Critic	อัลกอริทึมตระกูล off-policy · อ้างเพื่อเทียบ (.rsample()) เท่านั้น
SDK	Software Development Kit	ชุดโค้ดของ LimX สำหรับคุยกับหุ่นจริง — อยู่นอกขอบเขตไม้ดชุดนี้
PF · WF	PointFoot · WheelFoot	รุ่นเก่าอื่นของ Tron1 — ไม้ดชุดนี้ไม่ใช่ แต่ไฟล์ cfg ของมันชื่อซ้ำกับของ SF (หัวข้อ 0, 6)
SF	SoleFoot	รุ่นเก่าแบบของ Tron1 ที่ task Isaac-Limx-SF-Blind-Flat-v0 ใช้ · config ของมันคือ SF_TRON1AFlatPPOrunnerCfg (limx_rsl_rl_ppo_cfg.py:80)
SGD	Stochastic Gradient Descent	กฎอัปเดตแบบง่ายสุด "ไล่ลงเนิน" · สมการ (12)
SNR	Signal-to-Noise Ratio	อัตราส่วนสัญญาณต่อสัญญาณรบกวน — ใช้อธิบายขนาดก้าวของ Adam
TD	Temporal Difference	"ผลจริงก้าวนี้" ต่างจากที่ critic เดาเท่าไร · สมการ (19)
TRPO	Trust Region Policy Optimization	รุ่นพี่ของ PPO — บังคับระยะด้วยข้อจำกัด KL แทนการ clip
URDF	Unified Robot Description Format	ไฟล์บรรยายหุ่น (ROS) — ใช้เป็นอุปมาของ MDP ในไม้ด 05 §1.2
VAE	Variational AutoEncoder	สถาปัตยกรรม encoder แบบสุ่ม ซึ่ง repo นี้มี บั๊ก (is_vae = False, mlp_encoder.py:41)

ตัวย่อที่ **ไม่ทาง** เพราะเป็นชื่อสากลของสิ่งนั้น: tanh, exp, log, Box-Muller, ziggurat

2. หน่วยนับ

เรียงจากเล็กไปใหญ่ · หน่วยเหล่านี้คือหัววิธีหันตารางเดียวกัน (ไม้ด 07 §0)

หน่วย	ชนิด	ความหมาย	โค้ด	ค่าใน toy · Tron1	ใช้ ครั้งแรก
step (policy step)	หน่วยนับ	หนึ่งรอบการตัดสินใจ 0.02 s = 50 Hz · ข้างในมี physics 4 ก้าวย่อยที่ 200 Hz	decimation = 4 limx_base_env_cfg.py:497, sim.dt = 0.005 :501 · หนึ่ง step = alg.act แล้ว env.step on_policy_runner.py:175–177	1 ช่อง · เหมือนกัน	05 §1.1
transition	หน่วยนับ	ของทีหนึ่ง step ทั้งไว้ในตาราง: $(s, a, r, V, \log \pi_{old}, done)$	เก็บโดย add_transitions rollout_storage.py:131–148	1 ช่อง = 6 ค่าที่อัลกอริทึมใช้ (โค้ดเก็บ 12 ช่อง รวม μ, σ :145–146 ที่ KL ต้องใช้) · เหมือนกัน	05 §1.4
episode	หน่วยนับ	ช่วงต่อเนื่องของ step ใน env เดียว จนกว่าจะ done	—	env 1: $\{t_0, t_1, t_2\}$ จบที่ t2, episode ใหม่เริ่ม t3 · ยาวสูงสุด 1000 step	05 §1.4
done	เวกเตอร์ $(B,)$ (B = จำนวน env, สองแถวถัดไป)	ธง 1/0 ว่า step นี้เป็นก้าวสุดท้ายของ episode · เกิดจากสองสาเหตุที่ <i>อัลกอริทึมปฏิบัติต่างกัน</i> (แถวถัดไป)	done เก็บที่ rollout_storage.py:142 · ตัดสาย GAE ที่ :194	e1 t2 = 1 ที่เหลือ 0	05 §1.4

หน่วย	ชนิด	ความหมาย	โค้ด	ค่าใน toy · Tron1	ใช้ ครั้ง แรก
termination vs truncation	แนวคิด	<i>termination</i> = ล้ม (base_contact) — อนาคตถูกตัดจริง · <i>truncation</i> = หมดเวลา (time_out) — อนาคตยังมี แต่เราหยุดดู	TerminationsCfg limx_base_env_cfg.py:450-454 · time_outs ยัง bootstrap : บวก γV กลับเข้า reward ppo.py:163-168	toy มีแค่ termination (e1 t2)	05 \$1.4, \$4.5
env · <i>B</i>	หน่วยนับ	หุ่นจำลองหนึ่งตัว · <i>B</i> = จำนวนที่รับขนานกัน = จำนวนคอลัมน์ของตาราง	num_envs — ค่า CLI --num_envs 2048 (../README.md:265, ../experiments/exp_001_baseline_sf.md:28 — เทียบกับรากของ study_tron1/) · ค่าปริยายใน cfg คือ 4096 (limx_base_env_cfg.py:484)	2 · 2048	05 \$1.4
<i>T</i> จำนวน step ต่อ env	หน่วยนับ	จำนวนแถวของตาราง — เก็บที่ step ต่อ env ก่อนเรียนรู้หนึ่งครั้ง · ในสมการ (2) ตัว <i>T</i> หมายถึงก้าวสุดท้ายของ episode คณะความหมาย (หัวข้อ 7)	num_steps_per_env = 24 cfg:81	4 · 24	05 \$0.2
rollout	หน่วยนับ	ตารางเต็มหนึ่งใบ = <i>T</i> × <i>B</i> transition ที่เก็บก่อนเรียนรู้ · ไม้ตชุดนี้ เลิกใช้คำว่า "batch" สำหรับหน่วยนี้	num_steps_per_env = 24 cfg:81	4 × 2 = 8 · 24 × 2048 = 49,152	05 \$4.7
mini-batch · MB	หน่วยนับ	ช่องที่สุ่มมา $1/M$ ของตาราง → หนึ่ง gradient step	mini_batch_generator rollout_storage.py:222, สุ่มด้วย torch.randperm :230 · num_mini_batches = 4 cfg:99	4 ช่อง · 12,288	05 \$4.7
epoch	หน่วยนับ	เดินผ่านทุกช่องของตารางครบหนึ่งรอบ (สับ ครั้งเดียวต่อ iteration — torch.randperm rollout_storage.py:230 อยู่นอก loop epoch :251 ทุก epoch จึงใช้การค้นชุดเดิม)	num_learning_epochs = 5 cfg:98	2 · 5	05 \$4.7
gradient step	หน่วยนับ	หนึ่งครั้งของ zero_grad → backward → clip_grad_norm → step	ppo.py:287-290 · นับด้วย num_updates	2 × 2 = 4 · 5 × 4 = 20 ต่อ iteration	05 \$2.2
iteration	หน่วยนับ	เก็บ rollout (ช่วง A) → GAE (ช่วง B) → 20 gradient step (ช่วง C) → ล้างตาราง → เริ่มใหม่	ลูปที่ on_policy_runner.py:170 · GAE เรียกที่ :221-225 · update() ppo.py:180 · ล้างที่ :331	1 · 15,000 (cfg:82)	05 \$4.7

3. ปริมาณของโจทน์ — MDP

เรียงตามลำดับที่ไม้ต 05 พว

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ ครั้ง แรก	สูตร / ธรรมเนียม
$\mathcal{S}$ เซตของสถานะ (state space)	เซต	ทุกสถานการณ์ที่ หุ่นอาจอยู่ · ตัวใหญ่ = ทั้งเซต	obs 36 ตัวจาก PolicyCfg limx_base_env_cfg.py:130-146 · input จริงของ actor คือ 42 (หัวข้อ 4)	toy ไม้ต $\mathcal{S}$ ชัดเจน — "อะไรก็ตามที่ทำให้ critic พุด 5.0 และ actor พุด 0.20"	05 \$1.2	
s_t สถานะ ณ step t	เวกเตอร์	สมาชิกหนึ่งตัวของ $\mathcal{S}$		—	05 \$1.3	ตัวห้อย t = step ในตาราง เริ่ม 0 ใหม่ทุก iteration
$\mathcal{A}$ เซตของ action (action space)	เซต	ทุกท่าที่สั่งได้ · Tron1: $\mathbb{R}^8$	8 ข้อต่อ: ActionsCfg :116-121 ผ่าน joint_names=[".*"] → solefoot_cfg.py:41-57	$\mathbb{R}$ (1 มิติ) · $\mathbb{R}^8$	05 \$1.2	อ่านว่า "การแจกแจงบน $\mathcal{A}$ " ไม่ใช่ "ของ" — หัวข้อ 7
a_t action ที่สุ่มได้จริง	เวกเตอร์ (8,)	สมาชิกหนึ่งตัวของ $\mathcal{A}$ — ตัวแปรสุ่ม เปลี่ยนทุกครั้งที่ ทอย	self.distribution.sample() actor_critic.py:161	0.40	05 \$1.2	ไม้ตหน่วย จนกว่าจะ ×0.25 (สมการ 17)
$P(s' \mid s, a)$ transition	การแจกแจง	โลกตอนสอนยัง ไข่ — Tron1: PhysX	sim.dt :501, decimation :497	— (ตารางให้ V มาเลย ๆ)	05 \$1.2	ยึดตั้ง = "เมื่อรู้ว่า" — หัวข้อ 7
$R(s, a)$ ฟังก์ชันรางวัล (reward function)	ฟังก์ชัน	กติกาที่มนุษย์เขียน ให้คะแนน แต่ละ step · เขียนมือวงเล็บเสมอ เพื่อไม่ชนกับ R_t	RewardsCfg limx_base_env_cfg.py:368 — 20 พอนี	—	05 \$1.2	
r_t reward ที่ได้จริง	scalar	คะแนนของ step เดียว รู้ทันที	rewards rollout_storage.py:141	1.0	05 \$1.1	
γ discount	ค่าคงที่	อนาคตหนึ่งก้าวถัดไปมีค่าที่เปอร์เซ็นต์ของตอนนี้	gamma = 0.99 cfg:102 (ค่าปริยายของคลาส ppo.py:51 คือ 0.998 — ถูก cfg กับ)	0.9 · 0.99	05 \$1.2	γ^t · horizon ของ γ เอง $1/(1 - \gamma) = 10 \cdot 100$ step ; horizon ที่ GAE ใช้จริงคือ $1/(1 - \gamma\lambda)$ สมการ (34) — แถว λ หัวข้อ 5
Markov property	แนวคิด	อนาคตขึ้นกับสถานะปัจจุบัน อย่างเดียว ไม่ต้องดูประวัติ	ที่ขาด: base_lin_vel มีใน critic (:183) ไม่มีใน policy obs	toy ถือว่า Markov	05 \$1.3	สมการ (1)

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ ครั้ง แรก	สูตร / ธรรมเนียม
π policy (ทั่วไป) · π^* policy ที่ดีที่สุด	ฟังก์ชัน	กฎเลือก action จากสถานะ — "คำตอบ" ไม่ใช่ "โถกย" · ดาวหมายถึงตัวที่ทำให้ (2) สูงสุด			05 §1.2	คำนี้มีสามความหมาย — หัวข้อ 7
G_t return	scalar	ผลรวมรางวัลถ่วง γ ตั้งแต่ t จบจบ episode — ข้อเท็จจริง รู้ได้ต่อเมื่อ episode จบ	ไม่มีใน repo — Mean reward (on_policy_runner.py:200-206, :326) คือผลรวมต่อ episode ไม่ถ่วง γ จึงไม่ใช่ G_t · GAE ให้ R_t ไม่ใช่ G_t	env 1 episode แรก: $0.5 + 0.9(1.0) + 0.81(-2.0) = -0.22$	05 §1.5	$G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ — นิยาม ไม่มีเลขของตัวเอง · ใช้ใน (3), (7)
$V^\pi(s)$ value function	ฟังก์ชัน $\mathcal{S} \rightarrow \mathbb{R}$	ค่าทำนายของ G_t ถ้าเริ่มจาก s แล้วเดินตาม π	self.critic(...) actor_critic.py:170-172 คับ (B,1)	$V(s_0) = 5.0$ (env 0)	05 §1.6	สมการ (3), (7) · ด้วย $\pi =$ "ภายใต้ policy นี้"
$Q^\pi(s, a)$ action-value	ฟังก์ชัน $\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$	เหมือน V แต่เลือก action แรกไว้ก่อน	ไม่มีใน repo — Tron1 ไม่เคยคำนวณ Q · นิยามไว้เพราะ advantage ต้องใช้	ประมาณหนึ่งก้าว: $1.0 + 0.9(4.5) = 5.05$	05 §1.6	สมการ (5), (6)
Bellman equation	สมการ	V ของวันนี้ = รางวัลก้าวเดียว + $\gamma \times V$ ของพรุ่งนี้	ใช้เป็นเป้าใน compute_returns rollout_storage.py:195-199	$5.0 \approx 1.0 + 0.9(4.5) = 5.05$	05 §1.6	สมการ (4)
$A(s, a)$ advantage (นิยาม)	ฟังก์ชัน $\mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$	action นี้ดีกว่าค่าเฉลี่ยของ s เท่าไร · เขียนมีวงเล็บ หรือ A_t เสมอ ห้ามเขียน A ลอย ๆ	ไม่มี — ใช้ตัวประมาณ (25)-(26) แทน	$\approx 5.05 - 5.0 = +0.05$ (ประมาณหยาบ)	05 §1.7	$A = Q - V$ สมการ (8)
bootstrapping	แนวคิด	ใช้ค่าทำนายของตัวเองแทนอนาคตที่ยังไม่รู้	last_values ที่ขอบตาราง rollout_storage.py:190-191	$V_{last} = 3.0$ (env 0), 3.5 (env 1)	05 §1.6	
Monte Carlo vs TD	แนวคิด	MC รอผลจริงจนจบ (ไม่เอาเสียงแต่แกว่ง) · TD ใช้ก้าวเดียว + bootstrap (บึ่งแต่เอาเสียงตาม critic)		env 1 s_0 : MC -0.22 · TD $0.5 + 0.9(2.0) = 2.3$ · critic 3.0	05 §3.2	
tabular vs function approximator	แนวคิด	ตาราง (หนึ่งช่องต่อสถานะ) vs ฟังก์ชันที่สรุปรวม (NN)	ไม่มีตารางใน repo — RolloutStorage เก็บ transition ไม่ใช่ตารางสถานะ	สถานะของ toy เป็นจำนวนจริง → ตารางนี้แกว่งไม่จำกัด	05 §3.1	
on-policy / off-policy	แนวคิด	อก: ข้อมูลต้องมาจาก policy ตัวที่กำลังปรับ · off: ข้อมูลมาจาก policy อื่นได้ · PPO เป็น on-policy: ล้างตารางทุก iteration	OnPolicyRunner on_policy_runner.py:46 · storage.clear() ppo.py:331		05 §3.3	Sutton & Barto §5.4-5.5
importance sampling	แนวคิด	เอาตัวอย่างจากการแจกแจงหนึ่งไปประมาณค่าเฉลี่ยได้อีกการแจกแจง โดยถ่วงน้ำหนักด้วยอัตราส่วนความหนาแน่น — ที่มาของ ρ			05 §3.3	สมการ (20) · S&B §5.5
explore / exploit · ϵ -greedy	แนวคิด	ลองของใหม่ / ใช้ของที่รู้ว่าดี · ϵ -greedy (เขียนอักษรละติน) เป็นวิธีของ Q-learning คนจะเร่ร่อนกับ ϵ ในโน้ตนี้	Gaussian ทำสองอย่างพร้อมกันด้วย σ		05 §2.4	
deterministic vs stochastic policy	แนวคิด	คืน action ตง ๆ vs คืนการแจกแจงแล้วสุ่ม	act_inference vs act actor_critic.py:159-168		05 §2.1	$a = \mu_\theta(s)$ vs $a \sim \pi_\theta(\cdot s)$

4. ของใน policy และ neural network

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ครั้งแรก	สูตร / รสรมเนียน
θ พารามิเตอร์ของ actor	เวกเตอร์ยาว	weight ทั้งหมดที่การเรียนรู้ไปเปลี่ยน = actor 187,272 + logstd 8 = 187,280	actor_actor_critic.py:69-90, logstd :118	toy ไม่มี NN — μ ให้มาเฉย ๆ	05 \$2.1	
ϕ พารามิเตอร์ของ critic	เวกเตอร์ยาว	282,625 ตัว	actor_critic.py:92-111	—	05 \$2.1	
ψ พารามิเตอร์ของ encoder	เวกเตอร์ยาว	125,699 ตัว · มี optimizer แยก และถูก .detach() ออกจากกราฟของ loss หลัก	mlp_encoder.py:39, optimizer ppo.py:88-89, loss :313-321	—	05 \$4.4	actor+critic+l = 469,905 · รวม encoder = 595,604
node / weight / bias / activation / layer	แนวคิด	หน่วยคำนวณหนึ่งตัว: ผลรวมถ่วงน้ำหนัก + ค่าชดเชย แล้วผ่านฟังก์ชันกระตุ้น	nn.Linear สลับ activation actor_critic.py:71-90 · ชั้นซ่อน [512, 256, 128] cfg:88-89 · ชั้นสุดท้ายไม่มี activation actor_critic.py:77	—	05 \$2.2	สมการ (10)
ELU	ฟังก์ชัน	x ถ้า $x > 0$, $e^x - 1$ ถ้าไม่ · ไม่ "ตาย" ทางลบเหมือน ReLU และไม่อื่นตัวทางบวกเหมือน tanh	get_activation actor_critic.py:175-177	—	05 \$2.2	สมการ (11)
parameter vs hyperparameter	แนวคิด	parameter = ตัวเลขที่ optimizer เขียนกับ (θ, ϕ, ψ) · hyperparameter = ตัวเลขที่มนุษย์ตั้งใน config และไม่ถูกเรียนรู้ (หัวข้อ 6)			05 \$2.2	
o observation ของ policy	เวกเตอร์ (36,)	สิ่งที่เซนเซอร์ให้: ang_vel 3 + gravity 3 + joint_pos 8 + joint_vel 8 + last_action 8 + gait_phase 2 + gait_command 4	PolicyCfg limx_base_env_cfg.py:130-146	—	05 \$1.2	$12 + 3 \times 8 = 36$
c commands	เวกเตอร์ (3,)	ความเร็วเป้าหมาย (v_x, v_y, ω_z)		—	05 \$2.2	
h obsHistory	เวกเตอร์ (360,)	o ย้อนหลัง 10 step ต่อกัน · h หมายถึงสิ่งนี้เท่านั้น (หัวข้อ 7)	HistoryObsCfg :154-176 (history_length = 10 :175) · obs_history_len = 10 cfg:106 · $360 = 10 \times 36$ on_policy_runner.py:63	—	05 \$4.4	
$\hat{v}$ latent — ค่าประมาณ base_lin_vel	เวกเตอร์ (3,)	สิ่งที่ encoder เอาจาก h · เดิมไม่ตเรียกว่า z — เปลี่ยนชื่อเป็น $\hat{v}$ เพราะ loss ของมันคือ MSE กับ base_lin_vel จริง	encode() mlp_encoder.py:95-100 (.detach() เมื่อ output_detach = True cfg:109) · loss ppo.py:313-315	—	05 \$4.4	
s input ของ actor	เวกเตอร์ (42,)	[$\hat{v}$; o ; c] ต่อกัน	torch.cat ppo.py:137-139	—	05 \$2.2	$3 + 36 + 3 = 42$
critic input	เวกเตอร์ (230,)	privileged obs 227 + commands 3 · มีของที่หุ่นจริงวัดไม่ได้ (asymmetric actor-critic)	ppo.py:134 · base_lin_vel limx_base_env_cfg.py:183	—	05 \$2.1	
π_θ policy ตัวนี้	ฟังก์ชัน $\mathcal{S} \rightarrow \Delta(\mathcal{A})$	รับ s คืนการแจกแจงบน $\mathcal{A}$ — ไม่ได้คืน action · ประกอบด้วย ① NN ② logstd ③ สูตร $\mathcal{N}$ · ไม่มีรีโอบเจกต์เดียวในโค้ดที่เท่ากับมัน (self.actor_critic มี critic ปนอยู่ด้วย)	①+②+③ ใน actor_critic.py:155-157		05 \$2.1	สมการ (9)
$\Delta(\mathcal{A})$ simplex	เซต	เซตของการแจกแจงทั้งหมดบน $\mathcal{A}$ — "ระบึงทุกใบที่วางบน $\mathcal{A}$ ได้"			05 \$2.1	คนละตัวกับ δ_t และ $\Delta\mu$ — หัวข้อ 7
$\mu_\theta(s)$ จุดกลาง (mean)	เวกเตอร์ (8,)	output ของ NN = ค่าที่ actor คิดว่าดีที่สุด · นิพจน์ คำนวณใหม่ทุกครั้ง	mean = self.actor(observations) actor_critic.py:156	0.20	05 \$2.1	

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ครั้งแรก	สูตร / รสรมเนียน
l logstd	เวกเตอร์ (8,)	ลูกบิดคุมความกว้าง หนึ่งในตัวต่อข้อต่อ · ตัวแปร ที่นอนอยู่ข้าง ๆ NN ไม่ใช่ข้างใน ไม่รับ input · เปลี่ยนค่าได้ทางเดียวคือ optimizer.step() เขียนกับ · ไม่ใช่ L	nn.Parameter(torch.zeros(num_actions)) actor_critic.py:118	ln 0.5 = -0.6931 (คงที่ใน toy) · Tron1 เริ่ม 0	05 \$2.1	$\sigma = e^l$ สมการ (9)
σ ส่วนเบี่ยงเบนมาตรฐาน (standard deviation)	เวกเตอร์ (8,) broadcast → (B, 8)	ความกว้างของระฆัง · ไม่ขึ้นกับ σ ทุก state ทุก env ใช้ชุดเดียวกัน · ไม่ได้มาจาก critic · ไม่ใช่ความแปรปรวน (σ^2)	torch.exp(self.logstd) actor_critic.py:157 · ค่าเฉลี่ยขึ้นล็อก Policy/mean_noise_std on_policy_runner.py:269, :284	0.5 คงที่ · Tron1 เริ่ม 1.0 แล้วลด	05 \$2.1	ใน torch.distributions.Normal ช่องนี้ชื่อ scale · ไม้ตชุดนี้เขียน $\mathcal{N}(\mu, \sigma)$ ตำราเขียน $\mathcal{N}(\mu, \sigma^2)$ — หัวข้อ 7
$\pi_\theta(\cdot s)$ ระวัง 1 ใน	การแจกแจง	ผลของการใส่ s เข้า π_θ · จุดกลม = ช่องที่ยังไม่เต็ม · สร้างใหม่และทิ้งทุกสแต็ป ไม่มี weight	self.distribution = Normal(mean, ...) actor_critic.py:157 — ฉีบบเจกต์ ไม่ใช่ตัวเลข	$\mathcal{N}(0.20, 0.5)$	05 \$2.1	$\pi_\theta(\cdot s) = \mathcal{N}(\mu_\theta(s), \sigma)$ สมการ (9)
ϵ noise มาตรฐาน	เวกเตอร์ (8,)	เลขสุ่มจาก $\mathcal{N}(0, 1)$ ที่ทำให้ $a \neq \mu \cdot \epsilon$ หมายถึงสิ่งนั้นเท่านั้น (หัวข้อ 7)	ข้างใน torch.normal	+0.40	05 \$2.4	$a = \mu + \sigma \odot \epsilon$ สมการ (16)
$\odot$ Hadamard	ตัวดำเนินการ	คูณทีละตัวตรงตำแหน่ง — ในโค้ดคือ *			05 \$2.4	ไม่ใช่ dot product
$a \sim \pi_\theta(\cdot s)$ สุ่ม	โค้ด → เวกเตอร์	tilde อ่านว่า "ถูกสุ่มออกมาจาก" · เรียกซ้ำได้ค่าใหม่ — ไม่ใช่ฟังก์ชัน	.sample() actor_critic.py:161 (ได้ no_grad; torch.normal(loc, scale)) · เทียบ .rsample() ที่เขียน $\mu + \sigma \epsilon$ ตรงตัว — Tron1 ไม่ใช่	0.40	05 \$2.4	
z_i ระยะมาตรฐาน	scalar ต่อมิติ	a_i ห่างจาก μ_i ที่เท่าของ $\sigma_i \cdot z$ ตัวเดียวห้ามใช้ เรียก latent อีก		0.40	05 \$2.3	$z_i = (a_i - \mu_i)/\sigma_i$ · $\partial \log \pi / \partial \ell_i = z_i^2 - 1$
$\pi_\theta(a s)$ ความหนาแน่น ณ จุด a	scalar	ความสูงของระฆัง ณ a · เกิน 1 ได้ เพราะสิ่งที่ต้องเป็น 1 คือพื้นที่ ไม่ใช่ความสูง	.log_prob(a).sum(dim=-1) actor_critic.py:164 (คืน log)	0.7365 · ยอดที่ $\sigma = 0.5$ คือ 0.7979 · ที่ $\sigma = 0.25$ ยอด 1.596	05 \$2.3	สมการ (13)
$\log \pi_\theta(a s)$	scalar ต่อ env (B,)	log ของความหนาแน่นรวมทุกข้อต่อ · เทียบสอง buffer เป็น $\log \pi_{old}$	torch/distributions/normal.py:80-92 แล้ว .sum(dim=-1) actor_critic.py:164 · เทียบ ppo.py:147-149	-0.3058 (1 บิต)	05 \$2.3	สมการ (14), (15)
act vs act_inference	โค้ด	ตอนเทรน: สร้างระฆังแล้วสุ่ม · ตอน deploy: ค้น μ ตรง ๆ ไม่มี σ ไม่มีการสุ่ม	actor_critic.py:159-161 vs :166-168 · get_inference_policy on_policy_runner.py:376		05 \$2.6	deploy $\neq$ " $\sigma = 0$ " — σ ถูกทิ้ง
scale 0.25 · default offset	ค่าคงที่	แปลง action ไร้หน่วยเป็นเรเดียนรอบค่าเริ่มต้น	scale=0.25, use_default_offset=True limx_base_env_cfg.py:119-120 · ดูณ-บวกที่ IsaacLab/.../joint_actions.py:134 · offset = default_joint_pos :155-156 = 0.0 ทุกข้อ (solefoot_cfg.py:32-34)	0.40 → 0.10 rad (5.7°)	05 \$2.5	สมการ (17)
PD controller · k_p, k_d	ฟังก์ชัน	แรงบิด = k_p × ความคลาด ณ - k_d × ความเร็ว · ทำงาน 200 Hz	stiffness=45.0, damping=1.5 (บา) solefoot_cfg.py:50-51, damping=0.8 (ข้อเท้า) :61-62, effort_limit=80 :48, :59 · ส่งเป้าที่ joint_actions.py:160	—	05 \$2.5	สมการ (18)
gradient descent	แนวคิด/สมการ	หมุนลูกบิดส่วนทางความชันของ L	รูปจริงคือ Adam ppo.py:85		05 \$2.2	สมการ (12)
backpropagation · chain rule	แนวคิด	คำนวณ $\partial L / \partial (\text{ทุก ๆ กนู กบิ ด})$ ย้อนกลับตามกราฟที่ forward บันทึกรไว้ = loss.backward()	ppo.py:288		05 \$2.2	
autograd	โค้ด	ระบบหาอนุพันธ์อัตโนมัติของ PyTorch — สนแค่ว่าตัวเลขนี้ไหลในนิพจน์ที่นำไปสู่ L ไหม ไม่สนว่าเป็นเลขหรือหรือเปล่า			06 \$3.4	
Jacobian	แนวคิด	เมทริกซ์ของอนุพันธ์ย่อยของฟังก์ชันหลายมิติ · ใช้ตอนเปลี่ยนตัวแปรในความหนาแน่น			06 \$3.4	$\det \partial \epsilon / \partial a = 1 / \prod_i \sigma_i$
universal approximation	แนวคิด	NN ที่ใหญ่พอประมาณฟังก์ชันต่อเนื่องใดก็ได้ — เหตุผลที่ actor "ควง" ให้ action ตรง ๆ ได้ แต่ RL ต้องการการแจกแจงด้วยเหตุผลอื่น			06 \$3.5.0	

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ครั้ง แรก	สูตร / ธรรมเนียม
supervised learning	แนวคิด	เรียนจากคู่ (input, เดลย) · critic เรียนแบบนี้ แต่เดลยของมันมีตัวมันเองปนอยู่ (bootstrap)	value loss ppo.py:263-271		06 §3.4	
logits	แนวคิด	output ดิบก่อน softmax ของ head แบบ categorical · Tron1 ไม่มี — action ต่อเนื่อง			06 §3.5.0	
<code>.detach()</code> · <code>no_grad</code> · <code>nn.Parameter</code> · <code>requires_grad</code> · <code>.grad</code>	โค้ด	ตัดกราฟ / ไม่บันทึกกราฟ / tensor ที่ optimizer เห็น / รงว่าอยากได้ grad / ช่องข้าง ๆ ที่ <code>backward()</code> เดิน	a ถูก detach ppo.py:139 · <code>logstd</code> เป็น Parameter actor_critic.py:118		06 §3.4	<code>.grad</code> เป็นคนละกล่องกับค่า — หัวข้อ 5
reparameterization · <code>.rsample()</code>	แนวคิด	เขียนการสุ่มเป็น $\mu + \sigma$ เพื่อให้ gradient ไหลผ่าน σ ได้ · PPO ไม่ต้องใช้ (gradient ไหลผ่าน $\log \pi$) · SAC เลือกใช้เพราะ variance ต่ำกว่า	torch/distributions/normal.py:75-78 — ไม่มีใน repo		06 §3.5.4	

5. ของใน PPO loss และการเรียนรู้

เรียงตาม ลำดับการคำนวณ = rollout_storage.py:187-207 แล้ว ppo.py:205-290

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	คำใน toy	ใช้ ครั้ง แรก	สูตร / ธรรมเนียม
V_{last} bootstrap ที่ขอบตาราง	เวกเตอร์ (B,1)	critic ทำนายจากสถานะหลัง step สุดท้าย เพราะแถว T ไม่มีใครให้ดู	compute_returns(last_values) rollout_storage.py:190-191, เรียกจาก ppo.py:176-178, on_policy_runner.py:221-225	3.0 (env 0) · 3.5 (env 1)	05 §3.2	
mask (1 — done_t)	เวกเตอร์	ตัดพจน์ bootstrap ที่เมื่อ episode จบที่ช่องนี้	next_is_not_terminal rollout_storage.py:194	1 ทุกช่องยกเว้น e1 t2 = 0	05 §3.2	
δ_t TD error	scalar ต่อช่อง	ผลจริงที่ตัวนี้ + $\gamma \times$ ทำนายพรุ่งนี้ — ทำนายวันนี้ = critic พลาดเท่าไร	delta rollout_storage.py:195-199	+0.05 · e1 t2 = -3.00 (ถ้าไม่มี mask จะได้ +0.6 ซึ่งผิด)	05 §3.2	สมการ (19)
λ GAE lambda	ค่าคงที่	เชื่อ critic แค่ไหน vs รอผลจริง (0 = TD ล้วน, 1 = MC ล้วน)	lam = 0.95 cfg:103	0.8 · 0.95	05 §4.5	$\gamma\lambda = 0.72$ · 0.9405
A_t advantage (ตัวประมาณ GAE, ยังไม่ normalize)	scalar ต่อช่อง	ผลรวมถ่วงน้ำหนัก $(\gamma\lambda)^k$ ของ δ ในอนาคต หยอดที่ขอบ episode	recursion rollout_storage.py:200	-0.2722 · e1 t0 = -2.3272	05 §4.5	สมการ (25) พวจน $\equiv$ (26) recursion
R_t return เป้าของ critic (returns)	scalar ต่อช่อง	$A_t + V_t$ — ไม่ใช่ G_t มีค่าทำนายของ critic ปนอยู่	self.returns[step] rollout_storage.py:201	4.7278	05 §4.5	สมการ (27)
$\hat{A}_t$ advantage ที่ normalize แล้ว (advantages)	scalar ต่อช่อง	A ลบค่าเฉลี่ยหารด้วยส่วนเบี่ยงเบนของ ทั้งตาราง · std ใช้ $n - 1$ · ตัวที่ loss ใช้จริงคือตัวนี้	rollout_storage.py:204-207	+0.7975 · mean(A) = -1.1904, std = 1.1513 · e1 t1 = -0.9290	05 §4.5	สมการ (28)
policy gradient	สมการ	ทิศที่ต้องขยับ θ เพื่อให้ผลตอบแทนคาดหวังสูงขึ้น = ค่าเฉลี่ยของ $\nabla \log \pi \cdot A$	ไม่มี REINFORCE ใน repo — autograd ผ่าน actor_critic.py:164 → ppo.py:288 ทำหน้าที่นี้		05 §4.1	สมการ (22), (23) · S&B §13.2–13.4
$\partial \log \pi / \partial \mu_i$	scalar ต่อมิติ	$\log \pi$ ไล่ต่อ μ แคไหน		(0.40 — 0.20)/0.25 = 0.80	05 §4.1	สมการ (24)
surrogate	แนวคิด	ฟังก์ชันตัวแทนของเป้าหมายจริงที่คำนวณไม่ได้จากข้อมูลเก่า · ความชันตรงกับของจริงเฉพาะแถว ๆ θ_{old} — จึงต้อง clip			05 §4.6	

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ ครั้ง แรก	สูตร / บรรณนิยม
ρ_t probability ratio	scalar ต่อช่อง	policy ใหม่ขอ action เท่านั้นมากขึ้นที่ เท่า $\cdot \boldsymbol{a}$ ตัวเดิมจาก buffer ถูกตามระดั สองใบ	<code>log π_{new}</code> ppo.py:212-214 (หลัง <code>act()</code> ที่ :205 สร้างระดัใหม่แล้วโยน sample ที่จ) <code>ratio</code> :252-254	1.0356 (μ ขยับ +0.05) $\cdot$ e0 t2 / e1 t1 = 1.0779	05 \$3.3	สมการ (21) $\cdot$ = 1 ที่ gradient step แรก
ϵ ขอบ clip	ค่าคงที่	ρ ขยับได้ไม่เกิน $\pm 20\%$ ต่อ rollout $\cdot$ ϵ หมายถึงสิ่งนี้ เท่านั้น	<code>clip_param = 0.2</code> cfg:96	0.2	05 \$4.6	
L^{CLIP} surrogate loss	scalar	$-\hat{A}\rho$ กับ $-\hat{A} \text{ clip}(\rho)$ เอา ตัวที่ ใหญ่กว่า (แย กว่า) แล้วเฉลี่ยทั้ง minibatch $\cdot$ โค้ด เดิมลบไว้แล้ว เปเปอร์เขียน max ของ min ไม่มีลบ	ppo.py:256-260	ช่อง e0 t0 = $-0.8259 \cdot mb_0$ เฉลี่ย = -0.2958	05 \$4.6	สมการ (29) $\cdot$ clip ปิด gradient เฉพาะ ทิศที่ถูกและโหวต — โน้ต 07 \$5.3
L_V value loss	scalar	critic ทำนายพลาด จาก R_t เท่าไร (กำลัง สอง) — แบบ clipped: ใช้ตัวที่ใหญ่ กว่าระหว่างพลาดจริง กับพลาดของค่าที่ถูก หั่น	ppo.py:263-271 $\cdot$ <code>use_clipped_value_loss = True</code> cfg:95	e0 t0 (V_new 4.5): 0.0519	05 \$4.6	สมการ (32)
H entropy	scalar	ความไม่แน่นอนของ ระดั $\cdot$ ขึ้นกับ σ อย่าง เดียว ไม่เกี่ยวกับ μ $\cdot$ รวม 8 มิติแล้วเฉลี่ย minibatch	actor_critic.py:151-153, ppo.py:273 $\cdot$ สูตร <code>torch/distributions/normal.py:104-</code> <code>105</code>	0.7258 ต่อมิติ $\cdot$ $\times 8 =$ 5.8063	05 \$4.6	สมการ (31) $\cdot$ $\partial H / \partial \ell_i = 1$
c_V, c_H สัมประสิทธิ์	ค่าคงที่	น้ำหนักของ L_V และ H ใน loss รวม	<code>value_loss_coef = 1.0</code> cfg:94, <code>entropy_coef = 0.01</code> cfg:97 (ค่าปริยาย ของคลาส ppo.py:54 คือ 0.0 — ถูก cfg ก็น)	1.0 $\cdot$ 0.01	05 \$4.6	
L loss รวม	scalar ตัวเดียว ทั้งระบบ	เข้มวัดความแย ยัง น้อยยิ่งดี $\cdot$ ไม่ใช่ ℓ $\cdot$ KL ไม่อยู่ ในนี้	ppo.py:274-278	—	05 \$2.2	$L = L^{CLIP} + c_V L_V - c_H H$ สมการ (33)
$\nabla_{\theta} L \cdot$ <code>loss.backward()</code>	เวกเตอร์ ยาว / โค้ด	ความชันของ L เทียบ กับลูกบิตทุกตัว (469,905 ตัวใน optimizer หลัก) $\cdot$ เขียนลงช่อง <code>.grad</code> ไม่แต่ค่า	ppo.py:288		05 \$2.2	
<code>max_grad_norm</code> $\cdot$ <code>clip_grad_norm</code>	ค่าคงที่ / โค้ด	ถ้า norm ของ gradient ทั้งก้อน เกิน 1.0 ย่อลงตาม สัดส่วน — เขียนกับ <code>.grad</code> ก่อน <code>step()</code> เห็น	ppo.py:289 $\cdot$ <code>cfg:105</code>	1.0	05 \$4.6	ครอ actor+critic+ ℓ $\cdot$ encoder แยก (ppo.py:319-321)
$\alpha \cdot lr$ learning rate	scalar (ปรับได้)	ก้าวยาวแค่ไหน $\cdot$ hyperparameter — เริ่มจากที่มนุษย์ พิมพ์ แล้ว KL ปรับ เอง	<code>learning_rate = 1.0e-3</code> cfg:100 $\cdot$ ใต้ Adam ppo.py:85	0.1 ใน toy (เพื่อให้เห็นเลข) $\cdot$ 1e-3	05 \$2.2	สมการ (12), (37)
SGD	สมการ	$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla L$ — ถ้าแปรตามขนาด ความชัน	ไม่ได้ใช้จริง — ใช้เป็นรูปอ่านก็ศ		05 \$2.2	สมการ (12)
Adam $\cdot$ $\hat{m}, \hat{v} \cdot$ eps	สมการ	SGD ที่หาขนาด ความชันทั้ง (อัตราส่วนสัญญาณ/ สัญญาณรบกวน) — ก้าวราว α เมื่อ ความ ชันยืทางเดิม สม่ำเสมอ น้อยกว่า นั้นมากเมื่อ $\cdot$ <code>eps</code> = 1e-8 ค่าปริยาย torch	<code>optim.Adam(...)</code> ppo.py:85 $\cdot$ เขียนกับที่ <code>torch/optim/adam.py (param.addcdiv_)</code>		05 \$2.2	สมการ (36)

คำ / สัญลักษณ์	ชนิด	ความหมาย	โค้ด	ค่าใน toy	ใช้ ครั้ง แรก	สูตร / ธรรมเนียม
<code>optimizer.step()</code>	โค้ด	ที่เกี่ยวกับตัวเลขของ actor, critic, ℓ เปลี่ยนค่าระหว่าง <code>train</code> · อ่าน <code>.grad</code> แล้วเขียนกับ · ไม่ได้คำนวณ gradient	<code>ppo.py:290</code> · <code>encoder</code> แยก :321		05 \$2.2	
$\partial L / \partial \mu$	scalar ต่อมิติ	ก็ศที่ μ ต้องขยับ: เข้าหา $\hat{A}$ ที่ $\hat{A} > 0$ หนีจาก $\hat{A}$ ที่ $\hat{A} < 0$	<code>autograd</code>	-0.4955 (บี p)	05 \$4.1	สมการ (24) → ไม่ต 07 (G)
$\partial L / \partial \ell$	scalar ต่อมิติ	ก็ศที่ σ ต้องขยับ: กว้างขึ้นเมื่อค่าได้อยู่ไกล / ทำให้อยู่ใกล้ · entropy ดันกว้างคงที่	<code>autograd</code>	ที่ step แรก (p=1): surrogate +0.6699, entropy -0.01, รวม +0.6599 → ด้วย lr จริง 10^{-3} (ไม่ใช่ 0.1 ของ toy): ℓ -0.6931 → -0.693807, σ → 0.499670	06 \$3.4	สมการ (35)
KL $D_{KL}(\pi_{old} \parallel \pi_{new})$	ฟังก์ชัน (การแจกแจง, การแจกแจง) → scalar ≥ 0	ระยังใหม่ห่างจากระยังที่ใช้เก็บข้อมูลรอบนี้ แต่โหนด (ระยะสะสมตั้งแต่ต้น <code>update()</code> ไม่ใช่ก้าวล่าสุด) · ไม่สมมาตร · หน่วย nat · ไม่อยู่ใน loss	รายมิติแล้วบวก 8 มิติ <code>ppo.py:223-231</code> (บี +1e-5 ได้ log) → <code>kl_mean</code> :233 → กฎ lr :236-244 · ขึ้นชื่อเป็น Policy/mean_kl (คนละตัวแปร, :295, <code>on_policy_runner.py:285</code>)	$\Delta\mu = 0.05$: 0.0050 · $\Delta\mu = 0.15$: 0.045	05 \$4.6	สมการ (30) · ถ้า σ เท่ากัน เหลือ $(\Delta\mu)^2 / (2\sigma^2)$
<code>desired_kl · schedule</code>	ค่าคงที่	อุณหภูมิเป้าหมายของ thermostat · <code>adaptive</code> = เปิดกฎ	<code>cfg:104</code> , <code>cfg:101</code>	0.01	05 \$4.6	
กฎปรับ lr	สมการ	KL > 0.02 → lr ÷ 1.5 (พื้น 1e-5) · KL < 0.005 → lr × 1.5 (เพดาน 1e-2) · เช็ก ทุก minibatch (20 ครั้ง/iteration)	<code>ppo.py:236-244</code>	0.0050 → คงเดิม · 0.045 → ÷1.5	05 \$4.6	สมการ (37) · จาก 1e-3 ขนเพดานใน 6 ครั้ง ขนพื้นใน 12 ครั้ง
<code>early_stop · anneal_lr</code>	ค่าคงที่	ทั้งคู่ ปิด (ค่าปริยาย) · <code>anneal</code> ถ้าเปิดจะเป็นพื้นเลื่อยภายใน iteration ไม่ไ่ลดตามเวลา	<code>ppo.py:63-64</code> · กิ่ง :246-249, :280-284		06 \$3.4	
<code>extra_optimizer · est_learning_rate</code>	โค้ด / ค่าคงที่	optimizer แยกของ <code>encoder · lr</code> ของมัน ไม่ถูก KL และ	<code>ppo.py:61</code> (ค่าปริยาย 1e-3 ไม่ได้ตั้งใน <code>cfg</code>), :88-89, <code>.step()</code> :321		06 \$3.4	
<code>encoder loss</code>	scalar	MSE ระหว่าง $\hat{y}$ กับ <code>base_lin_vel</code> จริง (3 ช่องแรกของ <code>critic obs</code>)	<code>ppo.py:313-315</code>		05 \$4.4	

6. ค่าคงที่จาก config

หนึ่งแถวต่อบรรทัดของ block ที่รันจริง `SF_TRON1AFlatPPORunnerCfg` (`limx_rsl_rl_ppo_cfg.py:80-114`) · ไฟล์เดียวกันมี block ของหุ่นรุ่นอื่น (`PFPPORunnerCfg` :10-38, `PF` :42-76, `WF` :119-153) ซึ่ง**ทั้งบรรทัดและค่าบางตัวต่างกัน** (เช่น `max_iterations` 2000 / 10000, `save_interval` 200) — อ้างบรรทัดใน block นี้เท่านั้น

บรรทัด	ชื่อ	ค่า Tron1	ค่าใน toy	หมายถึง	อธิบาย ที่
<code>cfg:81</code>	<code>num_steps_per_env</code>	24	4	จำนวนแถวของตาราง (T')	05 \$4.7
<code>cfg:82</code>	<code>max_iterations</code>	15,000	1	จำนวนตารางทั้งหมด	05 \$1.5
<code>cfg:83</code>	<code>save_interval</code>	500	—	บันทึก checkpoint ทุกที่ iteration	05 \$2.6
<code>cfg:87</code>	<code>init_noise_std</code>	1.0	—	ดูเหมือนไม่ถูกใช้: บรรทัดที่ใช้บันทึกคอนเนนต์ก็ถึง (<code>actor_critic.py:117</code>) และ :118 เริ่ม ℓ ที่ 0 → $\sigma = 1.0$ อยู่แล้ว	05 \$2.1
<code>cfg:88-89</code>	<code>actor_hidden_dims · critic_hidden_dims</code>	[512, 256, 128]	—	ชั้นซ่อน	05 \$2.2
<code>cfg:90</code>	<code>activation</code>	"elu"	—	ฟังก์ชันกระตุ้น	05 \$2.2

บรรทัด	ชื่อ	ค่า Tron1	ค่าใน toy	หมายถึง	อธิบาย ที่
cfg:94	value_loss_coef	1.0	1.0	ϵ_V	05 \$4.6
cfg:95	use_clipped_value_loss	True	True	เปิด value clipping	05 \$4.6
cfg:96	clip_param	0.2	0.2	ϵ — ใช้ทั้ง p clip และ value clip	05 \$4.6
cfg:97	entropy_coef	0.01	0.01	ϵ_H	05 \$4.6
cfg:98	num_learning_epochs	5	2	epoch ต่อ iteration	05 \$4.7
cfg:99	num_mini_batches	4	2	หั่นตารางที่ก้อน	05 \$4.7
cfg:100	learning_rate	1e-3	0.1 (SGD เพื่อดูเลข)	α เริ่มต้น	05 \$2.2
cfg:101	schedule	"adaptive"	—	เปิดกฎ KL	05 \$4.6
cfg:102	gamma	0.99	0.9	γ	05 \$1.2
cfg:103	lam	0.95	0.8	λ	05 \$4.5
cfg:104	desired_kl	0.01	0.01	เป้าหมายของ thermostat	05 \$4.6
cfg:105	max_grad_norm	1.0	—	เพดาน norm ของ gradient	05 \$4.6
cfg:106	obs_history_len	10	—	ความยาวประวัติ $\rightarrow$ 360	05 \$4.4
cfg:109-111	output_detach · num_output_dim · hidden_dims (encoder)	True · 3 · [256, 128]	—	encoder ตัดจากกราฟหลัก · คืน $\hat{\mathbf{v}}$ 3 ค่า	05 \$4.4
limx_base_env_cfg.py:119-120	scale · use_default_offset	0.25 · True	—	action $\rightarrow$ เรเดียม	05 \$2.5
limx_base_env_cfg.py:497-501	decimation · episode_length_s · sim.dt	4 · 20.0 · 0.005	—	50 Hz policy / 200 Hz physics / 1000 step ต่อ episode	05 \$1.4
cfg/SF/limx_base_env_cfg.py:484	num_envs (คำปรีาย)	4096	2	ถูกกับด้วย CLI <code>--num_envs 2048</code>	05 \$4.7
solefoot_cfg.py:48-62	stiffness · damping · effort_limit	45 · 1.5 (บาท) / 0.8 (ข้อเท้า) · 80	—	PD gains	05 \$2.5
ppo.py:61, 63-64	est_learning_rate · early_stop · anneal_lr	1e-3 · False · False	—	คำปรีายของคลาส ไม่ได้ตั้งใน cfg	06 \$3.4

7. ระวังสับสน — ตัวอักษรซ้ำ คำซ้ำ

ตัวอักษร / คำ	ความหมายที่ 1	ความหมายที่ 2 (3)	กฎของไม้ตายนี้
ϵ vs ϵ vs <code>eps</code>	ϵ = ขอบ clip 0.2	ϵ = noise มาตรฐาน $\mathcal{N}(0, 1)$ · <code>eps</code> = ตัวกินหารศูนย์ใด ๆ (Adam 1e-8, normalize 1e-8, KL 1e-5)	สามตัวอักษร สามความหมาย · ϵ -greedy เขียนอักษร ละติน
T	สมการ (2): ทำสุดท้ายของ episode (ไม่รู้ล่วงหน้า)	หัวข้อ 2 และสมการ (26): จำนวนแถวของตาราง = <code>num_steps_per_env</code> 24 ลงที่ $\cdot \mathbf{A}_T = \mathbf{0}$ ที่ขอบตาราง	ตัวเดียวกันสองความหมายตามบริบท — ในตาราง 2×4 หมายถึงแถว
$\mathbf{r}_t$ vs $\mathbf{R}_{t+1}$	ไม้ตายนี้: reward ของช่อง ($\mathbf{s}_t, \mathbf{a}_t$) เรียก $\mathbf{r}_t$ ตาม <code>rollout_storage</code>	Sutton & Barto เรียกตัวเดียวกันว่า $\mathbf{R}_{t+1}$ (ได้รับหลักทำ)	เวลาเทียบกับตำรา เลื่อนดัชนี 1
L vs ℓ	L = loss, scalar ตัวเดียวทั้งระบบ, เข็ม	ℓ = <code>logstd</code> , เวกเตอร์ 8 ตัว, ลูกบิณฑ์	ลูกบิณฑ์ไปขยับเข็ม ไม่ใช่ของชนิดเดียวกัน · L^{CLIP}, L_V = ตัวยกบอกชนิด ตัวห้อยบอก network
π	policy	3.14159 ใน $\sqrt{2\pi}$	ดูบริบท — ถ้ามีตัวห้อย θ หรือวงเล็บ คือ policy
π_θ vs $\pi_\theta(\cdot \mid \mathbf{s})$ vs $\pi_\theta(\mathbf{a} \mid \mathbf{s})$	ฟังก์ชัน	การแจกแจง · scalar	สามจุดบนสายชนิด — คนละของ
$R(\mathbf{s}, \mathbf{a})$ vs G_t vs R_t	ฟังก์ชันรางวัล (กติกาก)	return จริง (ข้อเท็จจริง) · <code>returns</code> = $\mathbf{A}_t + \mathbf{V}_t$ (เป้าหมาย critic มี critic ใด)	เขียนวงเล็บ / ตัวห้อยเสมอ ห้าม $\mathbf{R}$ ลอย ๆ
$\mathcal{A}$ vs $\mathbf{A}_t$ vs $\hat{\mathbf{A}}_t$	เซตของ action	advantage ใด · advantage ที่ normalize (ตัวที่ loss ใช้)	ห้าม $\mathbf{A}$ ลอย ๆ
$\Delta(\mathcal{A})$ vs δ_t vs $\Delta\mu$	simplex	TD error · ระยะที่ μ ขยับ	ตัวใหญ่มีวงเล็บ = simplex · ตัวเล็กมีตัวห้อย = TD
$\sigma: \mathcal{N}(\mu, \sigma)$ vs $\mathcal{N}(\mu, \sigma^2)$	ไม้ตายนี้: ช่องสอง = ส่วนเบี่ยงเบนมาตรฐาน (ตาม <code>scale</code> ของ torch)	ตำรา: ช่องสอง = ความแปรปรวน	ทุกครั้งที่ว่าตำรา เติม "(σ^2 ในตำรา)"
$\mathbf{z}$	เดิม = latent ของ encoder	$\mathbf{z}_i$ = ระยะมาตรฐาน $(\mathbf{a}_i - \mu_i)/\sigma_i$	latent เปลี่ยนชื่อเป็น $\hat{\mathbf{v}}$ · $\mathbf{z}$ เหลือความหมายเดียว

ตัวอักษร / คำ	ความหมายที่ 1	ความหมายที่ 2 (3)	กฎของไม้ตกนี้
h	obsHistory (B, 360)	เดิม ขึ้นของพลาทงกลาง	ขึ้นผลต่างเขียน $\Delta \ell \cdot h$ เหลือความหมายเดียว
①②③ vs ทิศ vs ช่วง	①②③ = สามชั้นของ policy (NN, logstd, สูตร..)	ทิศทางการแปลง = ทิศ ก / ข / ค · ช่วงของวัตถุในโดอะแกรม = 1–4	สามระบบตัวเลข สามชนิดของสิ่ง
B vs "batch" vs MB	B = num_envs 2048	"batch" เล็กใช้ สำหรับ 49,152 → เรียก rollout · MB = 12,288	
α	learning rate	(ใน Beta(α, β) ของตัวราสถิติ)	ไม้ตกนี้ใช้ความหมายแรกเท่านั้น
"step"	policy step 0.02 s	env.step() · optimizer.step() · physics sub-step 0.005 s	ระบุทุกครั้งว่า step ของอะไร
"clip"	ρ clip ที่ [0.8, 1.2]	value clip ที่ ±0.2 · clip_grad_norm_ ที่ 1.0	สองอันแรกใช้ clip_param ตัวเดียวกัน อันที่สามคนละเลข
"mean"	μ จุดกลาง	.mean() เปลี่ยนทั้ง minibatch · Mean_reward ในล็อก	
"policy"	ฟังก์ชัน S → Δ(A) (ตัวรา)	NN ที่ deploy (ไม่แถมเมอร์) · ไฟล์ policy.onnx	สามความหมาย ตรงกับตอน deploy เท่านั้น — ตอนเทรน NN เดียว ๆ ยังไม่ใช่ policy
"KL"	ปริมาณ D_{KL}(p q)	ตัวเลข Policy/mean_kl ในล็อก · "KL penalty" (พอนิใน loss — Tron1 ไม่มี)	
"การแจกแจงบน" vs "ของ"	บน A (พื้นที่ สุ่มไม่ได้)	ของ a (ตัวแปรสุ่ม กอยใหม่ได้)	ถามว่า "สิ่งนี้สุ่มได้ไหม"
ตัวห้อย t	step ในตาราง เริ่ม 0 ใหม่ทุก iteration	(ในตัวรา: เวลาต่อเนื่องใน episode)	ไม้ตกนี้ใช้ความหมายแรก
kl_mean vs mean_kl	ค่าที่คูณ lr (ppo.py : 233)	ค่าเฉลี่ย 20 รอบที่ขึ้น tensorboard เท่านั้น (: 295)	

8. ตัวอย่างเดียวกันที่ใช้ทั้งสามไม้ตก

ตารางจั่ว 2 env × 4 step, action 1 มิติ — เลิกพอเขียนทุกเลขลงกระดาษ · ไม้ตก 05 หยิบทีละช่อง ไม้ตก 06 ขยายเป็น 8 มิติเฉพาะจุดที่ shape สำคัญ ไม้ตก 07 ใส่ทั้ง iteration · ตัวเลขทุกตัวข้างล่าง คำนวณด้วยสคริปต์เดียวและตรวจซ้ำแล้ว

8.1 ค่าคงที่

ค่าคงที่	toy	Tron1
B · T · มิติของ a	2 · 4 · 1	2048 · 24 · 8
γ · λ · γλ	0.9 · 0.8 · 0.72	0.99 · 0.95 · 0.9405
σ	0.5 คงที่ (ℓ = −0.6931)	เริ่ม 1.0 แล้วเรียนรู้
ε (clip)	0.2	0.2
epoch × minibatch	2 × 2 = 4 gradient step	5 × 4 = 20
การยับยั้ง μ ที่ใช้ลาอิต	step แรก: μ_{new} = μ_{old} + 0.05 ทุกช่อง · กรณี clip: μ_{new} = μ_{old} + sign(A) sign(a − μ_{old}) · 0.15 (ทิศที่ (G) สั่ง)	—

8.2 ข้อมูลดิบ — สิ่งที่ NN คาย (μ, V) · ลูกเต๋าของ policy (ε) · สิ่งในโลกให้มา (r, done)

		t=0	t=1	t=2	t=3	V _{last}
env 0	μ	0.20	0.30	0.10	0.40	
	ε	+0.40	−0.20	+0.80	0.00	
	r	1.0	0.5	1.0	0.5	
	V	5.0	4.5	4.0	3.5	3.0
	done	0	0	0	0	
env 1	μ	−0.10	0.00	0.20	−0.30	
	ε	−0.60	+0.80	+0.20	+0.40	
	r	0.5	1.0	−2.0	0.0	
	V	3.0	2.0	1.0	4.0	3.5
	done	0	0	1	0	

ออกแบบให้ครบทุกกรณี: env 1 ล้มที่ t2 (done ตัดสาย GAE, episode ใหม่เริ่ม t3) · **e0 t2 กับ e1 t1 เป็นคู่แฝด** (**|a − μ| = 0.40** เท่านั้น, **A** คนละเครื่องหมาย) ไม้ตก clip ทั้งสี่กรณี

8.3 เฉลย — ทุกค่าที่ derive ได้จาก 8.2

ช่อง	a	log π _{old}	δ _t	A _t	R _t	A _t	log π _{new} (+0.05)	ρ	L _i	∂L/∂μ
e0 t0	0.40	−0.3058	+0.0500	−0.2722	4.7278	+0.7975	−0.2708	1.0356	−0.8259	−0.4955
e0 t1	0.20	−0.2458	−0.4000	−0.4475	4.0525	+0.6452	−0.2708	0.9753	−0.6293	+0.3776
e0 t2	0.50	−0.5458	+0.1500	−0.0660	3.9340	+0.9766	−0.4708	1.0779	−1.0526	−1.4737

ช่อง							(+0.05)			
e0 t3	0.40	-0.2258	-0.3000	-0.3000	3.2000	+0.7733	-0.2308	0.9950	-0.7695	+0.1539
e1 t0	-0.40	-0.4058	-0.7000	-2.3272	0.6728	-0.9874	-0.4708	0.9371	+0.9253	-1.2954
e1 t1	0.40	-0.5458	-0.1000	-2.2600	-0.2600	-0.9290	-0.4708	1.0779	+1.0014	+1.4020
e1 t2	0.30	-0.2458	-3.0000	-3.0000	-2.0000	-1.5718	-0.2308	1.0151	+1.5955	+0.3191
e1 t3	-0.10	-0.3058	-0.8500	-0.8500	3.1500	+0.2956	-0.2708	1.0356	-0.3062	-0.1837

mean(**A**) = -1.1904 · std_{n-1}(**A**) = 1.1513 · mb₀ = {e0t0, e0t2, e1t1, e1t3} เผลี่ย **L^{CLIP}** = **-0.2958** · mb₁ เผลี่ย = +0.2805 · **H** = 0.7258 · KL(Δμ = 0.05) = 0.0050

8.4 การ์ดของมาตรฐาน — e0 t0 (ค่าที่คอลัมน์ "ค่าใน toy" ใช้)

ปริมาณ	ค่า	มาจาก
μ, ε, σ	0.20, +0.40, 0.5	ข้อมูลดิบ
$a = \mu + \sigma \varepsilon$	0.20 + 0.5 × 0.40 = 0.40	(16)
$z = (a - \mu) / \sigma$	0.40	
$\pi(a \mid s) \cdot \log \pi$	0.7365 · -0.3058	(13), (14) — ยอด ระงับที่ σ = 0.5 คือ 0.7979
$r, V, V(s_1)$	1.0, 5.0, 4.5	ข้อมูลดิบ
Bellman ตรวจ	1.0 + 0.9 × 4.5 = 5.05 vs 5.0	(4) — ส่วนต่างคือ δ
$\delta \cdot A \cdot R \cdot \hat{A}$	+0.05 · -0.2722 · 4.7278 · +0.7975	(19), (26), (27), (28)
หลัง μ ขยับ +0.05: $\log \pi_{new} \cdot \rho \cdot L_i \cdot \partial L / \partial \mu$	-0.2708 · 1.0356 · -0.8259 · -0.4955 → SGD α = 0.1: μ 0.25 → 0.2995	(21), (29), (24)
ที่ gradient step แรก (p = 1): $\partial L / \partial \ell$	surrogate -$\hat{A}(z^2 - 1) = +0.6699$ · entropy -0.01 · รวม +0.6599 → ถ้าด้วย lr จริงของ Tron1 α = 10⁻³ (ไม่ใช่ 0.1 ของแควบน — เพื่อให้เห็นว่าถ้าจริงเล็กแค่ไหน; ส่วนอื่นของ toy ตรง σ ไร่): ℓ -0.6931 → -0.693807 → σ 0.499670	(35)
ถ้า critic ขยับเป็น $V_{new} = 4.5$ (สมมติ): L_V	$V_{clip} = 5.0 + \text{clip}(4.5 - 5.0, \pm 0.2) = 4.8$ · $\max((4.5 - 4.7278)^2, (4.8 - 4.7278)^2) = \max(0.0519, 0.0052) = 0.0519$	(32)

8.5 ช่องอื่นที่โดนหยิบใช้

ช่อง	ใช้ทำอะไร	ค่าสำคัญ
e1 t1 (คู่แฝดของ e0 t2)	action แย้ ที่ระงับ เดียวกัน	a = 0.40, $\hat{A} = -0.9290$, $\log \pi_{old} = -0.5458$
e0 t2	action ดี ที่ระงับ เดียวกัน	a = 0.50, $\hat{A} = +0.9766$, $\log \pi_{old} = -0.5458$
คู่แฝดกับกฎ ±0.15	clip คนสี่กรณี	ขยับ ถูกทาง 0.15: p = 1.2153 (e0 t2) / 0.7520 (e1 t1) → clip, gradient 0 · ขยับ ผิดทาง 0.15: p เท่ากันแต่ max เลือกพจน์ดิบ gradient ไท -1.6157 / +1.1291 · KL = 0.045 > 0.02 → lr ÷ 1.5
e1 t2 (done = 1)	mask ตัด bootstrap	δ = -3.00 · ถ้าไม่มี mask จะได้ +0.6 ("ล้มแล้วตีกว่าที่คาด" — ผิด)
คอลัมน์ env 1	episode ที่จบจริง	G₀ = -0.22 vs V(s₀) = 3.0 · TD ถ้าเดียว = 2.3
e0 t3	ε = 0 → a = μ	log π = -0.2258 = ยอดระงับ

8.6 การขยายเป็น 8 มิติ (ใช้เฉพาะโน้ต 06 §3.5.5 และ §3.5.7)

ข้อต่อที่ 1 ≡ e0 t0 ส่วนข้อต่อ 2-8 เป็นเลขอิสระ:

μ = [0.20, − 0.30, 0.10, 0.40, − 0.10, 0.60, − 0.50, 0.30] **ε** = [0.40, − 0.20, 0.60, 0.00, − 0.60, 1.20, − 0.80, 0.20]
a = [0.40, −0.40, 0.40, 0.40, −0.40, 1.20, −0.90, 0.40] ·
log **N_i** = [−0.3058, −0.2458, −0.4058, −0.2258, −0.4058, −0.9458, −0.5458, −0.2458] · **Σ** = **-3.3263** · **π** = **0.03592** · μ ขยับ
+0.05 ทุกมิติ: **Σ log π_{new} = -3.2863** · ρ = 1.0408 · **L^{CLIP} = -0.8300** (ที่ **$\hat{A} = +0.7975$**) ·
∂L/∂μ_i = [−0.498, +0.498, −0.830, +0.166, +1.162, −1.826, +1.494, −0.166] · **H = 5.8063** · KL = 0.040

8.7 เลขที่เลิกใช้แล้ว

σ = 0.3, π = 1.088, ρ = 1.22, 0.1525 rad (โน้ต 05 เดิม) · **μ = 0.42, $\hat{A} = +1.10$, log π = -3.3955, ρ = 1.0953**, 0.03352 (โน้ต 06 เดิม) ·
V = 650, δ = 28.15, ρ = 1.25, L_V = 781.2 (walkthrough ต้นทาง — เหลือเป็นตัวอย่างขนาดจริงในโน้ต 07 §6 และ errata ในโน้ต 06 §4 เท่านั้น)

9. ดัชนีสมการ

เลขเดียวกันทั้งชุด เรียงตามลำดับที่โน้ต 05 พหุ · (35)–(37) เป็นของโน้ต 06 · โน้ต 07 ใช้เลขเดียวกัน

#	สมการ	อ่านว่า	derive ที่
(1)	$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_t, a_t, s_{t-1}, \dots)$	Markov: อดีตไม่เพิ่มข้อมูล	05 §1.3
(2)	$\pi^* = \arg \max_{\pi} \mathbb{E}[\sum_{t=0}^T \gamma^t r_t]$	เป้าหมายทั้งหมด	05 §1.5
(3)	$V^{\pi}(s) = \mathbb{E}_{\pi}[\sum_{k \geq 0} \gamma^k r_{t+k} \mid s_t = s]$	นิยาม value	05 §1.6
(4)	$V^{\pi}(s) = \mathbb{E}[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t = s]$	Bellman	05 §1.6
(5)	$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[\sum_{k \geq 0} \gamma^k r_{t+k} \mid s_t = s, a_t = a]$	นิยาม action-value	05 §1.6
(6)	$Q^{\pi}(s, a) = \mathbb{E}[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t = s, a_t = a]$	Q ถ้าวัดจริง	05 §1.6
(7)	$V^{\pi}(s) = \mathbb{E}[G_t \mid s_t = s]$	value = ทำนาย return	05 §1.7
(8)	$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$	advantage	05 §1.7
(9)	$\pi_{\theta}(\cdot \mid s) = \mathcal{N}(\mu_{\theta}(s), \sigma), \quad \sigma = e^{\ell}$	policy ของ Tron1	05 §2.1
(10)	$y = f(\sum_i w_i x_i + b)$	หนึ่ง node	05 §2.2
(11)	$\text{ELU}(x) = x \ (x > 0), \ e^x - 1 \ (x \leq 0)$	activation	05 §2.2
(12)	$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$	gradient descent	05 §2.2
(13)	$\pi(a \mid s) = \frac{1}{\sigma \sqrt{2\pi}} \exp(-\frac{(a-\mu)^2}{2\sigma^2})$	Gaussian PDF	05 §2.3
(14)	$\log \pi(a \mid s) = -\frac{1}{2} \ln 2\pi - \ln \sigma - \frac{(a-\mu)^2}{2\sigma^2}$	log-density สามพจน์	05 §2.3 · 07 §2.2
(15)	$\log \pi_{\theta}(a \mid s) = \sum_{i=1}^8 \log \pi(a_i \mid s)$	รวม 8 ข้อต่ออิสระ	05 §2.3
(16)	$a = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$	สุ่ม	05 §2.4
(17)	$q_{\text{target}} = 0.25 a + q_{\text{default}}$	action → ชุดเป้าหมาย	05 §2.5
(18)	$\tau = k_p(q_{\text{target}} - q) - k_d \dot{q}$	PD	05 §2.5
(19)	$\delta_t = r_t + (1 - \text{done}_t) \gamma V(s_{t+1}) - V(s_t)$	TD error (มี mask)	05 §3.2 · 07 §4.2
(20)	$\mathbb{E}_{a \sim p}[f(a)] = \mathbb{E}_{a \sim q}[\frac{p(a)}{q(a)} f(a)]$	importance sampling	05 §3.3
(21)	$\rho_t = \frac{\pi_{\text{new}}(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}})$	ratio	05 §3.3 · 07 §5.2
(22)	$\nabla_{\theta} \pi_{\theta} = \pi_{\theta} \nabla_{\theta} \log \pi_{\theta}$	score function	05 §4.1
(23)	$\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a \mid s) A(s, a)]$	policy gradient	05 §4.1
(24)	$\partial \log \pi / \partial \mu = (a - \mu) / \sigma^2$	ทิศของ μ	05 §4.1 · 07 §5.2
(25)	$A_t = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k}$	GAE พหุคูณ	05 §4.5
(26)	$A_t = \delta_t + (1 - \text{done}_t) \gamma \lambda A_{t+1}, \ A_T = 0$	GAE recursion (ไล่ถ)	05 §4.5 · 07 §4.3
(27)	$R_t = A_t + V_t$	เป้าหมายของ critic	05 §4.5 · 07 §4.4
(28)	$\hat{A} = (A - \bar{A}) / (\text{std}_{n-1}(A) + 10^{-8})$	normalize	05 §4.5 · 07 §4.5
(29)	$L^{\text{CLIP}} = \text{mean}[\max(-\rho \hat{A}, -\text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) \hat{A})]$	surrogate loss (รูปไล่ถ)	05 §4.6 · 07 §5.2
(30)	$D_{KL}(\mathcal{N}_o \parallel \mathcal{N}_n) = \sum_i [\ln \frac{\sigma_{n,i}}{\sigma_{o,i}} + \frac{\sigma_{o,i}^2 + (\mu_{o,i} - \mu_{n,i})^2}{2\sigma_{n,i}^2} - \frac{1}{2}]$	KL ของเกาส์เซียน	05 §4.6 · 07 §5.7
(31)	$H = \sum_i [\frac{1}{2} + \frac{1}{2} \ln 2\pi + \ln \sigma_i]$	entropy	05 §4.6 · 07 §5.5
(32)	$L_V = \text{mean}[\max((V - R)^2, (V_{\text{clip}} - R)^2)], \ V_{\text{clip}} = V_{\text{old}} + \text{clip}(V - V_{\text{old}}, \pm \epsilon)$	value loss	05 §4.6 · 07 §5.4
(33)	$L = L^{\text{CLIP}} + c_V L_V - c_H H$	loss รวม	05 §4.6 · 07 §5.6
(34)	$1/(1 - \gamma \lambda)$	horizon ที่ GAE มองเห็น	05 §4.7
(35)	$\partial L^{\text{CLIP}} / \partial \ell_i = -\hat{A} \rho (z_i^2 - 1), \quad \partial H / \partial \ell_i = 1$	ทิศของ σ	06 §3.4
(36)	$\theta \leftarrow \theta - \alpha \hat{m} / (\sqrt{\hat{v}} + \text{eps})$	Adam	06 §3.4
(37)	$\text{KL} > 2\text{-desired} \rightarrow \text{lr} \div 1.5 \cdot \text{KL} < \frac{1}{2}\text{-desired} \rightarrow \text{lr} \times 1.5 \cdot$ ในช่วง [1e-5, 1e-2]	กฎปรับ lr	06 §3.4
(G)	$\partial L / \partial \mu = -\hat{A} \rho (a - \mu_{\text{new}}) / \sigma^2$	(24) ต่อผ่าน ρ และ (29) — เลขของไม้ดัด 07	07 §5.2

ตารางแปลงเลขเก่า

ที่มา	เลขเก่า → ใหม่
ไม้ดัด 05 เดิม (1)–(12)	(1) → (2) · (2) → (3) · (3) → (4) · (4) → (7) · (5) → (10) · (6) → (12) · (7) → (13) · (8) → (16) · (9) → (18) · (10) → (8) · (11) → (25) · (12) → (29)
ไม้ดัด 07 เดิม	(7) → (14) · (10) → (19) · (11) → (26) · (4) → (27) · (G) คงเดิม
ไม้ดัด 06 ขึ้น E1–E15	E1–E2 ไม่มีสมการ (ต่อ tensor) · E3 ↔ (10) · E4 ↔ (9) · E5 ↔ (9) · E6 ↔ (16) · E7 ↔ (13) · E8 ↔ (14) · E9 ↔ (15) · E10–E12 ไม่มีสมการ (เก็บ/อ่าน buffer, จำนวนใหม่) · E13 ↔ (21) · E14 ↔ (29) · E15 ↔ (24)/(G)
walkthrough ต้นทาง	(1b) → (12) · (3) → (4) · (5) → (26) · (6) → (29)

ตรงกับโค้ดจริง 2026-09-11 · ตัวเลขทั้งหมดในหัวข้อ 8 จำนวนด้วย `../scripts/toy_answer_key.py` (รับ `python3 scripts/toy_answer_key.py` จากรากของ repo) ·
บรรจุใน `torch/` อ้างจาก `torch 2.5.1` ใน env `trn1` — เลขบรรทัดของไลบรารีขึ้นกับเวอร์ชัน